

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code	CSA12	Course Title	Computer Architecture for Multicore Systems
CO Assessed	CO5	Assessment	Assessment Tool 1 – Constraint-Based Problem Solving
Student Name	M.HARSHA	Reg. No.	192525108
Department	BTech AI & ML	Date	27/08/2026

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices

- Use software interrupts for background tasks

Code of Conduct:

I M.HARSHA(192525108) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M.HARSHA*

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1 – Real-Time Embedded I/O Communication Mechanism

Construct an I/O communication mechanism for a real-time embedded system with CPU utilization below 15%, multiple sensors generating interrupts every 1 ms, vectored interrupt handling, and a choice between memory-mapped and I/O-mapped I/O for optimal latency.

1.1 Constraint Analysis

- The CPU utilization contributed by I/O handling must remain below 15%; therefore, the interrupt service routine (ISR) must be extremely short.
- Each sensor can generate an interrupt every 1 ms, so the system must tolerate a high interrupt rate without losing events.
- Vectored interrupt handling is mandatory so that the processor can enter the correct ISR directly through an interrupt vector.
- The I/O addressing method must be selected for low-latency register access. Memory-mapped I/O is selected because device registers can be accessed through the normal load/store mechanism and can be efficiently integrated with the processor's memory system.
- Interrupt priority and nesting must prevent a less important sensor from delaying a time-critical sensor.

1.2 Proposed Architecture

Each sensor is connected to the interrupt controller and assigned a vector. The controller maintains priority information and sends the corresponding vector to the CPU. On entry, the ISR acknowledges the interrupt, reads only the minimum required status/data from the memory-mapped device register, stores the event in a RAM queue or circular buffer, and exits. More expensive processing is performed later by a deferred background task. This separates time-critical interrupt response from computationally expensive processing.

1.3 CPU Utilization Check

As an explicit design assumption, consider 8 sensors, each generating one interrupt every 1 ms. If the complete ISR overhead per interrupt is limited to 5 μ s, the worst-case interrupt workload is $8 \times 1000 \times 5 \mu\text{s} = 40,000 \mu\text{s}$ per second = 40 ms per second, or approximately 4% CPU utilization. This is comfortably below the 15% constraint. The design therefore has additional headroom for normal application processing and occasional interrupt bursts.

1.4 Interrupt and I/O Handling

1. Sensor raises its hardware interrupt line.
2. The interrupt controller identifies the source and priority and supplies the corresponding vector.
3. CPU saves the minimum context required for the ISR and jumps directly to the vectored handler.
4. The ISR acknowledges the device, performs the required memory-mapped register read/write, and places the event/data into a RAM queue.
5. The ISR clears the interrupt condition and returns quickly.
6. A deferred task processes queued sensor data outside the critical ISR path.

1.5 Justification

Memory-mapped I/O is preferred for this real-time design because it provides a uniform register-access mechanism using normal load/store instructions and avoids maintaining a separate I/O instruction space. Vectored interrupts reduce dispatch overhead, while short ISRs and deferred processing keep CPU

utilization and latency predictable. Priority-based interrupt handling ensures that time-critical sensors receive service first without allowing background work to consume the real-time CPU budget.

Question 2 – High-Speed NVMe I/O Subsystem with DMA

Design an I/O subsystem for storage throughput above 3 GB/s, using NVMe over PCIe, while preventing the CPU from handling bulk data transfer and integrating DMA with interrupt notification.

2.1 Constraint Analysis

- Required storage throughput is greater than 3 GB/s, so the interconnect must provide sufficient bandwidth with practical protocol overhead.
- NVMe over PCIe is mandatory; therefore, a PCIe-attached NVMe controller is selected.
- The CPU must not copy bulk storage data; DMA must transfer data directly between the NVMe device and system memory.
- Completion notification is required, so MSI/MSI-X-style interrupt notification can inform the CPU after DMA operations or command completions.

2.2 Proposed Architecture

A PCIe Gen4 $\times 4$ connection is selected to provide substantial bandwidth headroom above the 3 GB/s requirement. The NVMe controller uses submission and completion queues in system memory. The CPU prepares commands and buffer descriptors, while the NVMe controller/DMA path moves the actual payload between the SSD and RAM. After completion, the device generates an interrupt notification so the CPU only performs completion processing rather than copying the data.

2.3 Operation

7. CPU places an NVMe command and buffer information into the submission queue.
8. The NVMe controller fetches the command through the PCIe interface.
9. The controller transfers storage data directly into or out of RAM using DMA.
10. The completed operation is recorded in the completion queue.
11. The controller generates an interrupt notification, such as MSI-X, to alert the CPU.
12. CPU services the completion, updates software state, and submits additional commands.

2.4 Throughput and CPU Analysis

The design intentionally selects a link with more theoretical capacity than the 3 GB/s target rather than operating at the edge of the requirement. PCIe Gen4 $\times 4$ has enough link bandwidth for this target under practical conditions, while the NVMe queue-based design allows multiple commands to remain in flight. DMA eliminates repeated CPU memory copies, which is essential for sustaining high throughput without excessive CPU utilization.

2.5 Justification

NVMe is appropriate for high-performance storage because it is designed for PCIe and supports parallel command queues. PCIe Gen4 $\times 4$ gives the required bandwidth margin, DMA handles bulk movement, and interrupt notification informs the CPU only when software action is required. This directly satisfies all four constraints while keeping the CPU available for application work.

Question 3 – Multi-Device USB and Thunderbolt Communication Architecture

Construct a communication architecture for six USB peripherals and two high-speed Thunderbolt devices, avoiding interrupt starvation, using hardware and software interrupts, and ensuring fair bus arbitration.

3.1 Constraint Analysis

- Six USB peripherals must share the USB host subsystem, while two high-speed devices require Thunderbolt connectivity.
- High-speed devices must not monopolize the shared bus or CPU service path.
- Hardware interrupts are needed for urgent device/controller events; software interrupts or deferred software handlers can process less time-critical work.
- Fair arbitration is required so that every active device receives service within a bounded interval.

3.2 Proposed Architecture

The six USB peripherals are attached through a USB host controller, while the two high-speed devices use a Thunderbolt controller connected through the high-speed system interconnect. Traffic is maintained in separate queues. A fair scheduling policy, such as round-robin with finite service quotas, prevents a continuously active high-bandwidth device from starving other devices.

3.3 Interrupt Strategy

- Hardware interrupts are used for urgent events such as device completion, controller faults, link changes, or time-critical status conditions.
- Software interrupts/deferred handlers are used for protocol processing, packet handling, logging, and other work that does not require immediate hardware response.
- Interrupt priorities are configured so that critical controller events preempt less important background processing.
- Interrupt coalescing or batching can be used for high-rate completion events where the hardware supports it, reducing excessive interrupt overhead.

3.4 Fair Bus Arbitration

The arbiter gives each active queue a bounded service opportunity. A simple round-robin policy gives each requester a turn; weighted round-robin can be used when Thunderbolt devices legitimately require more bandwidth. A finite service quota prevents any one requester from occupying the bus indefinitely. Separate queues also prevent a slow or blocked device from holding up unrelated traffic.

3.5 Justification

The architecture satisfies the device-count constraint while separating high-speed traffic from ordinary USB traffic. Fair scheduling directly addresses starvation, and the combination of hardware interrupts with deferred software processing prevents the CPU from spending excessive time in interrupt context. This provides predictable service while still allowing high-speed devices to achieve higher throughput when bandwidth is available.

Question 4 – 500 MB/s Data Acquisition System

Design a data acquisition system that continuously receives 500 MB/s, minimizes CPU intervention, compares DMA with interrupt-driven I/O, and uses memory-mapped I/O for device registers.

4.1 Constraint Analysis

- The incoming stream is continuous at 500 MB/s, so the system must prevent buffer overrun and sustain the data rate.
- CPU intervention must be minimal; therefore, moving each sample using CPU instructions is unsuitable.
- DMA transfer must be compared with interrupt-driven I/O because the data rate is high.
- Device control/status registers must be accessed using memory-mapped I/O.

4.2 Proposed Architecture

The acquisition device exposes control and status registers through memory-mapped I/O. The CPU configures the DMA controller with source, destination, transfer length, and buffer information. The DMA engine then transfers incoming data directly into a circular or double buffer in RAM. The CPU is notified after a block completes rather than after every sample.

4.3 DMA versus Interrupt-Driven I/O

Criterion	DMA Transfer	Interrupt-Driven I/O	Best Choice
CPU involvement	Low; CPU configures and handles completions	High; CPU services many interrupts and performs transfers	DMA
500 MB/s suitability	High; bulk movement is offloaded	Poor for very frequent transfers	DMA
Latency	Low and predictable when buffers are pre-posted	Can vary with interrupt load	DMA
Efficiency	High for large blocks	Lower due to per-event overhead	DMA

4.4 Operation and Feasibility

13. CPU initializes the acquisition device through memory-mapped control registers.
14. CPU configures DMA descriptors for one or more RAM buffers.
15. The acquisition device produces the continuous 500 MB/s stream.
16. DMA transfers the stream into the active RAM buffer without CPU copying.
17. When a buffer reaches its configured boundary, DMA switches to the next buffer and sends a completion interrupt.
18. CPU processes completed buffers in batches while DMA continues filling the next buffer.

4.5 Justification

DMA is clearly preferable at 500 MB/s because interrupt-driven I/O would require the CPU to respond to a very large number of transfer events and would waste processor cycles on data movement. Memory-mapped I/O is still used for device registers because the CPU only needs occasional control/status accesses; DMA performs the high-volume payload transfer. A circular or double-buffer arrangement ensures continuous acquisition while the CPU processes previously captured blocks.

Question 5 – Nested Vectored Interrupt Handling under a 10 μ s Latency Constraint

Construct an interrupt handling mechanism that supports nested interrupts, guarantees maximum interrupt latency below 10 μ s, uses vectored interrupts for high-priority devices, and uses software interrupts for background tasks.

5.1 Constraint Analysis

- Nested interrupts are required so that a high-priority event can preempt a lower-priority ISR.
- Maximum interrupt latency must remain below 10 μ s, so interrupt entry, vector lookup, context saving, and higher-priority masking time must be tightly bounded.
- High-priority devices must use vectored interrupts for direct dispatch.
- Background tasks should use software interrupts/deferred execution so they do not block urgent hardware events.

5.2 Proposed Mechanism

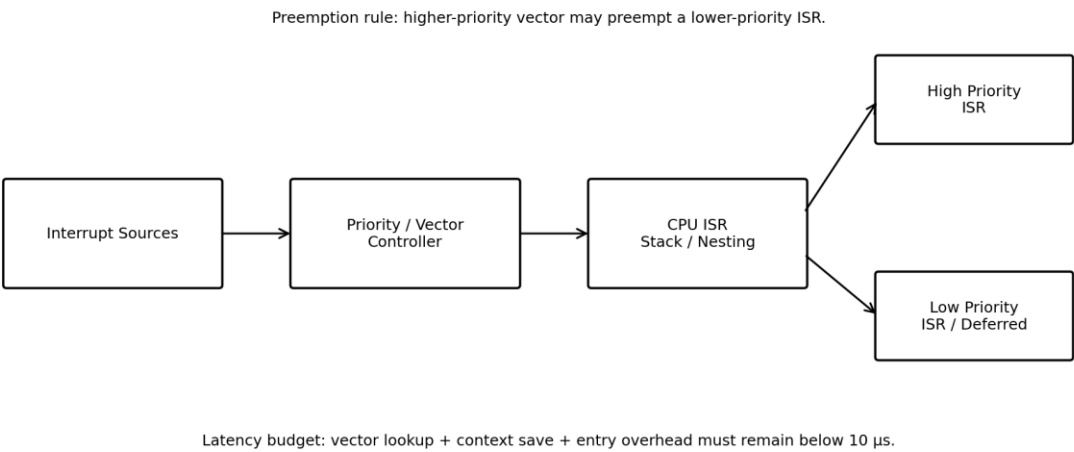


Figure 5. Nested vectored interrupt mechanism with priority-based preemption.

The interrupt controller assigns a vector and priority level to each interrupt source. When an interrupt arrives, the controller performs priority resolution. A high-priority interrupt can preempt a lower-priority ISR after the minimum required context has been saved. The processor's interrupt stack or equivalent context mechanism preserves the interrupted ISR state. After the high-priority ISR completes, execution resumes at the preempted ISR.

5.3 Latency Budget

Component	Target Budget	Design Action
Interrupt recognition + priority resolution	$\leq 2 \mu$ s	Dedicated interrupt controller and fixed priority resolution
Vector lookup + ISR entry	$\leq 2 \mu$ s	Vectored dispatch and fixed vector table
Minimum context save	$\leq 2 \mu$ s	Keep ISR entry context minimal
Highest-priority masking/critical section	$\leq 2 \mu$ s	Very short non-preemptible sections

Safety margin	$\geq 2 \mu\text{s}$	Reserve margin for hardware/platform variation
---------------	----------------------	--

The proposed budget totals 8 μs for the normal worst-case path, leaving approximately 2 μs of margin under the 10 μs maximum. The implementation must verify the actual platform timing using hardware measurement or cycle-accurate profiling; the budget is an engineering target, not a claim that every processor will automatically meet it.

5.4 Nested Interrupt Sequence

19. A low-priority hardware interrupt enters its vectored ISR.
20. The ISR saves the minimum context and begins servicing the device.
21. A higher-priority hardware interrupt arrives.
22. The interrupt controller permits preemption and the CPU saves the additional required context.
23. The CPU dispatches directly to the high-priority vector and executes its short ISR.
24. The high-priority ISR acknowledges/completes the urgent operation and returns.
25. The processor restores the lower-priority ISR context and resumes it.
26. Background work is triggered through software interrupts/deferred processing after the urgent hardware paths are clear.

5.5 Justification

Vectored interrupts minimize dispatch time, while priority-based nesting ensures that urgent devices are not delayed by lower-priority service. The strongest control on latency is to keep ISRs short, bound all non-preemptible sections, and defer background work. The explicit 8 μs target budget provides a 2 μs margin against the 10 μs constraint and makes the latency requirement measurable and testable.

Rubric Alignment – Level 4 / Exemplary

Rubric Criterion	How the Answers Address It	Weightage
Problem Understanding	Every question explicitly identifies the system objective and all stated constraints before proposing a design.	25%
Constraint Analysis	Constraints are prioritized around latency, CPU utilization, throughput, starvation, bandwidth, interrupt frequency, and CPU intervention.	25%
Solution Development	Each question includes a feasible architecture, operating sequence, numerical feasibility checks where appropriate, and a clear implementation strategy.	25%
Application of Concepts	The solutions apply vectored/hardware/software interrupts, memory-mapped I/O, DMA, PCIe, USB, NVMe,	15%

	Thunderbolt, queues, buffering, and bus arbitration as required by the assessment.	
Justification	Each design decision is followed by technical reasoning showing why it satisfies the constraints and why alternatives are less suitable.	10%

Conclusion

The five proposed architectures are designed directly around the constraints in the assessment brief. Real-time sensor communication uses vectored interrupts and memory-mapped registers with short ISRs; the NVMe subsystem uses PCIe and DMA to achieve high storage throughput without CPU data copying; the multi-device architecture uses fair arbitration and mixed hardware/software interrupt handling; the 500 MB/s acquisition system uses DMA and buffered transfers; and the nested interrupt mechanism uses priority-based vectored dispatch with a bounded latency budget. Together, these designs demonstrate the required CO5 concepts and provide explicit justification for the selected mechanisms.